QA agentizada

Más capacidad de prueba, con conocimiento compartido y evidencia.

QA

Una evolución del workbook que ya usamos
Para QA, Desarrollo, Negocio y gerentes

Conocimiento Pruebas Mantenimiento

Más trabajo cubierto con la capacidad disponible

El objetivo es cubrir más pruebas, o sostener la misma carga con menos dedicación del equipo.

Capacidad

Copilot ayuda a preparar, ejecutar y documentar trabajo repetible.

Agentizar un proceso

Darle a un asistente un objetivo, contexto y herramientas permitidas para realizar tareas encadenadas.

Cómo evaluar el resultado

Medir el trabajo aceptado, incluyendo revisión, retrabajo y mantenimiento.

> El ahorro y los cambios de dotación son objetivos a evaluar. Todavía no son resultados demostrados.

El workbook ya es nuestra base

Cada colaborador tiene GitHub Copilot en su PC. Ya hay instrucciones y procesos para usarlo en QA.

01

Diseñar casos

Copilot ayuda a transformar una necesidad en pasos y resultados esperados.

→ 02

Ejecutar pruebas

El workbook ya usa Playwright MCP para navegar la UI web.

→ 03

Entregar evidencia

Genera reportes y Markdown para reproducir y corregir bugs.

> El equipo de automatizaciones aporta los runners y las prácticas existentes. La propuesta amplía ese trabajo.

Un ciclo por aplicación, módulo o flujo

Podemos avanzar con pocos flujos críticos sin esperar a documentar toda la compañía.

Conocer → **Validar** → **Probar** → **Aprender**

Conocimiento

Notas técnicas conectadas y documentación funcional revisada.

Pruebas

Casos, automatizaciones y ejecuciones con evidencia.

Cambios

Features y bugs actualizan los documentos y las pruebas relacionadas.

> La unidad de avance es un flujo con entregables completos y una versión identificable.

Responsabilidades que se complementan

El agente prepara trabajo. Las personas aportan criterio y deciden lo que necesita validación.

Desarrollo	Aporta qué cambió, sus dependencias y los detalles de implementación.
QA y automatizaciones	Revisan cobertura y resultados, aceptan pruebas reutilizables y clasifican fallos.
Negocio o responsable del proceso	Confirma reglas críticas, condiciones y excepciones.
Responsable del conocimiento	Coordina revisión, publicación y sincronización. No reconstruye todo solo.
Copilot	Redacta, propone casos, ayuda a mantener pruebas y usa herramientas autorizadas.

Cada herramienta tiene una función

Boards, Test Plans, Repos y Pipelines se conectan, pero guardan cosas distintas.

B

Boards

Trabajo, features y bugs. Relaciones y seguimiento.

T

Test Plans

Casos, suites y planes. Cobertura y resultados asociados.

R

Azure Repos

Git: historial de documentos, código y pruebas.

P

Pipelines

Ejecuciones repetibles, resultados y artefactos.

> Git conserva revisiones exactas. Un runner es el programa que ejecuta las pruebas.

Primero, una base confiable

Pasos 1 a 4. Cada etapa deja algo concreto que la siguiente puede usar.

01 **Segundo cerebro técnico**

Copilot identifica fuentes, dependencias, comportamiento y dudas.

02 **Borrador funcional**

Describe procesos y resultados. Distingue lo observado de lo inferido.

03 **Validación focalizada**

QA, negocio o el responsable revisan las reglas críticas. Publicamos lo aprobado y mantenemos pendientes explícitos.

04 **Casos trazables**

Copilot propone y QA revisa resultados esperados y cobertura. Vinculamos regla y caso en Test Plans y Boards.

Después, pruebas que vuelven a servir

Pasos 5 a 8. El ciclo termina cuando el trabajo se puede repetir y mantener.

05 **Automatizaciones UI, API y base**

Preparación de datos, verificaciones y herramientas autorizadas.

06 **Ejecución y evidencia**

Guardamos resultados. Las pruebas aceptadas pasan a Pipelines.

07 **Reporte y seguimiento**

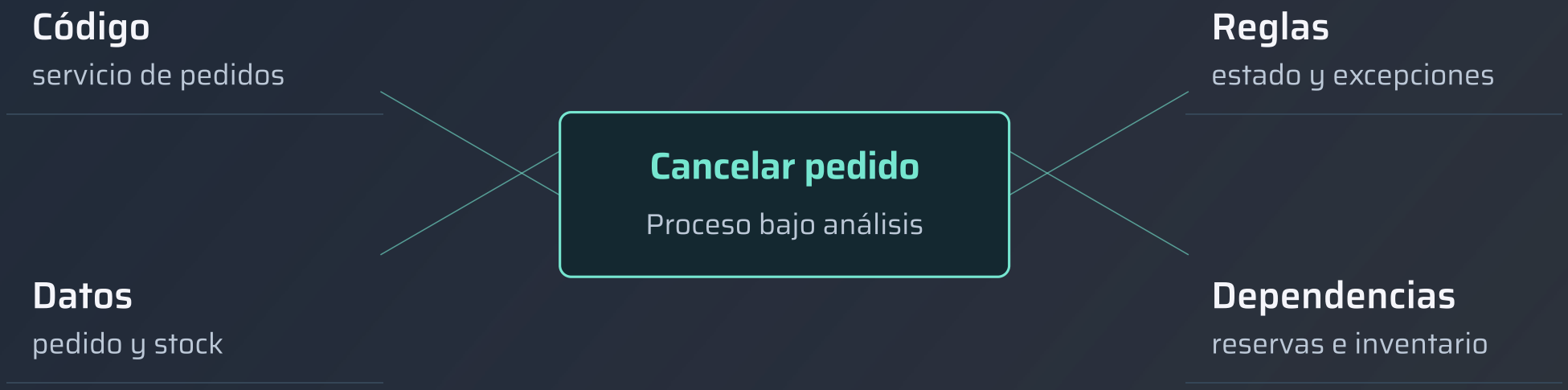
Clasificamos los fallos y registramos defectos confirmados con sus relaciones.

08 **Mantenimiento por cambio**

Actualizamos conocimiento, documentación, casos y automatizaciones impactadas.

Un segundo cerebro técnico

Notas Markdown (archivos de texto) pequeñas y conectadas que explican la aplicación y sus dependencias.



Las notas se enlazan entre sí y con sus fuentes. Cada nota conserva el contexto necesario para leerla sola.

Una nota que se puede verificar

La brevedad ayuda a encontrar información. La procedencia permite confiar en ella.

cancelacion-pedido.md

Ejemplo ilustrativo

Contenido

El servicio devuelve stock al cancelar un pedido pendiente.

Fuente y versión

Archivo / método + revisión Git.
Sistema 2.4, ambiente QA.

Relaciones

Pedido, reserva, inventario y casos asociados.

Estado de la afirmación

Validado: confirmado por responsable.
Observado: visto en código o ejecución.
Inferido: hipótesis a revisar.
Pendiente: falta resolver.

Validación

Rol responsable, fecha y dudas abiertas.

Se puede arrancar aunque falten documentos

El segundo cerebro se construye con el material y los accesos disponibles.

01

Reunir fuentes

Código, Swagger/OpenAPI, consultas, tickets y pruebas existentes.

→ 02

Reconstruir comportamiento

Copilot describe qué hace el sistema y dónde encontró cada afirmación.

→ 03

Redactar para negocio

Explica procesos, condiciones, resultados y excepciones en lenguaje funcional.

> La documentación funcional nace de esa base técnica. Las inferencias quedan visibles para revisión.

Del código a una regla explicable

Ejemplo ilustrativo: la cancelación de un pedido y su efecto sobre el stock.

COMPORTAMIENTO OBSERVADO

El código devuelve stock

Copilot encuentra una actualización de inventario cuando se cancela el pedido.

La nota enlaza el método y la versión analizada.

BORRADOR FUNCIONAL

¿Cuándo corresponde?

Describe estado permitido, precondiciones, resultado y excepciones.

Quedan dudas sobre pedidos despachados y reservas parciales.

> El código muestra lo implementado. Un conocedor del proceso confirma qué debería pasar.

Revisión humana donde aporta criterio

QA, negocio o el responsable revisan borradores cortos y resuelven las dudas críticas.

“¿Un pedido despachado puede cancelarse y devolver stock?”

1 **Confirmar la regla**
Alguien con conocimiento del proceso define las condiciones y excepciones.

2 **Aprobar la versión**
La documentación registra quién validó y qué quedó pendiente.

3 **Usar esa referencia**
Los casos toman el resultado esperado de la regla revisada.

> No hace falta escribir todo desde cero ni revisar manualmente cada corrida. La revisión se concentra en reglas y cambios.

Cuando nadie puede confirmar una regla

El conocimiento sigue siendo útil si su estado queda claro.

Observado

Conservamos lo que hace hoy el sistema, con fuente y versión.

Pruebas de caracterización

Registran el comportamiento actual y detectan si cambia.

Alcance del resultado

Sirven para regresión. La corrección funcional sigue pendiente de validación.

> Cada pendiente mantiene un responsable o un punto de resolución. La IA no convierte una observación en una regla aprobada.

Separar tareas mejora la revisión

Podés usar el mismo modelo en sesiones separadas o modelos distintos para sumar perspectivas.

20%

Dos modelos lo aceptan

La regla real permite 10%

Si ambos usan una fuente equivocada, pueden repetir el mismo error.

Cambiar de modelo no garantiza independencia.

> Separá responsabilidades y contrastá fuentes. Cada resultado esperado necesita respaldo revisado.

El buscador devuelve fragmentos relevantes

RAG es recuperar conocimiento para darle contexto al asistente. La respuesta depende de qué fragmentos encuentra.

Consulta con contexto

Aplicación, módulo, ambiente y versión del sistema. Estado de validación y revisión del documento.

Información completa para el caso

Regla, precondiciones, excepciones y dependencias relacionadas.

Validación en el piloto

Probar preguntas conocidas. Revisar si aparecen los fragmentos necesarios y si corresponden a la versión elegida.

Buscador compartido

RAGFlow es una opción para el MVP. Confirmar la implementación y verificar metadatos y filtros en la versión instalada.

> Recuperar un fragmento no garantiza comprender todo el proceso.

Git conserva la historia del conocimiento

Azure Repos es la fuente canónica de notas técnicas y documentación funcional.

01

Azure Repos

Markdown con historial, responsables y revisiones exactas.

→ 02

Publicación aprobada

Sincronización propuesta de una versión revisada, identificada por su revisión Git.

→ 03

Buscador compartido

Indexa esa publicación y recupera fragmentos con los mismos metadatos.

> Subir archivos al buscador no reemplaza el historial Git. No asumimos versionado Git nativo de RAGFlow.

Dos fuentes para diseñar una prueba

La documentación funcional dice qué debe pasar. El segundo cerebro técnico ayuda a comprobarlo.

REFERENCIA FUNCIONAL VALIDADA

Resultado esperado

“Al cancelar un pedido pendiente, se libera el stock reservado”.

+

CONOCIMIENTO TÉCNICO

Cómo verificarlo

UI o endpoint, estado del pedido, reserva e inventario.

Caso propuesto por Copilot, revisado por QA

Precondiciones, datos, pasos, verificaciones y cobertura de excepciones.

La relación regla y caso empieza al diseñar

Test Plans registra los casos. Boards los vincula con el trabajo que les da origen.

Documento aprobado

Regla R-07

Cancelación permitida

→

Azure Test Plans

Caso C-12

Datos + esperado

→

Azure Repos

Prueba A-12

Código reutilizable

→

Azure Pipelines

Corrida E-31

Versión + evidencia

La feature o el bug de Boards enlaza estos elementos. QA revisa resultados esperados y cobertura antes de automatizar.

La exploración deja una prueba reutilizable

Cada exploración automatizable termina en código que el equipo puede aceptar y repetir.

01 Preparación de datos y ambiente

02 Acciones y verificaciones explícitas

03 Evidencia y limpieza cuando corresponda

04 Versionado y revisión del equipo

Copilot crea y mantiene

Ayuda a convertir la exploración en una automatización estable.

El runner ejecuta

Pipelines repite las pruebas aceptadas. La regresión no depende de una nueva exploración libre del modelo en cada corrida.

Copilot y Azure DevOps MCP

MCP es una interfaz para que el asistente use herramientas y datos de un sistema.

GitHub Copilot

Cliente en modo agente

↔

Azure DevOps MCP

Servidor oficial, local o remoto

↔

- Boards
- Test Plans
- Azure Repos
- Pipelines

> Conexión propuesta. Verificar entorno/cliente, versión, autenticación, permisos y dominios habilitados. El catálogo no implica que toda operación esté disponible.

Este es el MCP de Azure DevOps. Azure MCP para recursos cloud tiene otro alcance.

La conexión que ya usa el workbook

Playwright MCP permite que Copilot navegue la interfaz web y ayude a explorar flujos.

Copilot
Interpreta el caso

→ **Playwright MCP**
Herramientas de navegador

→ **UI web**
Acciones y evidencia

Exploración asistida

Recorrer el flujo, identificar controles y observar resultados.

Regresión repetible

Guardar una prueba aceptada y versionada. El runner la vuelve a ejecutar.

> UI significa interfaz de usuario. El alcance de esta conexión es la web.

La metodología se adapta al sistema

Conviven aplicaciones actuales y sistemas antiguos (legacy) de unos diez años. La herramienta depende de la tecnología.

Web	Reusar Playwright y las automatizaciones de navegador existentes.
Servicios / API	Probar endpoints y reglas con el runner o cliente de API apropiado.
Base de datos	Comprobar estados y efectos con consultas SELECT controladas.
Escritorio y otros legacy	Relevar tecnología y automatizaciones existentes. Elegir una herramienta compatible.

> Priorizamos flujos críticos y regresión. Playwright para web no es una promesa de automatizar aplicaciones de escritorio.

Swagger describe el contrato técnico

OpenAPI define puntos de acceso (endpoints), entradas y respuestas. La prueba también verifica el negocio.

200

HTTP exitoso

La solicitud respondió.
Eso es una señal técnica.

Cancelar un pedido

¿Cambió al estado correcto?

¿Se liberó el stock que correspondía?

¿Se respetaron estados prohibidos y excepciones?

¿Los valores y efectos coinciden con la regla validada?

> API es la interfaz entre sistemas. Una respuesta exitosa no alcanza para demostrar corrección funcional.

Operaciones SQL con un alcance definido

SQL es el lenguaje para consultar datos. El agente elige una operación aprobada. El ejecutor conserva el SQL fijo.

01

Copilot solicita

consultar_estado_pedido con un identificador válido.

→ 02

Ejecutor controlado

Valida parámetros, ambiente permitido y operación del catálogo.

→ 03

Base responde

Conexión con lectura mínima, límites de tiempo y cantidad de resultados.

> Puede ser un script o CLI, o una herramienta MCP. MCP ofrece la interfaz. Los permisos se imponen también en el ejecutor y la base.

Ejemplo: consultar el estado de un pedido

La operación responde una pregunta de QA sin habilitar SQL arbitrario.

SOLICITUD DEL AGENTE

```
consultar_estado_pedido
  pedido_id: 1042
  ambiente: QA
```

El ejecutor valida el identificador y el ambiente antes de consultar.

SQL FIJO EN EL EJECUTOR

```
SELECT id, estado
FROM pedidos
WHERE id = :pedido_id
```

El parámetro se vincula de forma segura, sin concatenar SQL.

Resultado ilustrativo id: 1042 estado: cancelado

> QA compara el estado con el esperado. Otras operaciones aprobadas pueden verificar reserva y stock.

El control habilita el uso del agente

El límite funciona cuando las restricciones existen fuera de la conversación.

Ejecutor protegido

El agente no modifica las operaciones aprobadas ni accede a credenciales por otra vía.

El catálogo define parámetros, ambiente, tiempos y cantidad de filas.

Conexión y base

Permisos mínimos de lectura y acceso acotado a los datos necesarios.

Si el agente puede editar el script o usar otra conexión, puede saltear el límite del catálogo.

> Los controles permiten automatizar consultas concretas con un alcance verificable. Un prompt, por sí solo, no impone permisos.

Una corrida que se puede repetir

Pipelines ejecuta las automatizaciones aceptadas con el runner que el equipo ya utiliza.

01

Fijar la corrida

Versión del sistema, ambiente, revisión de conocimiento y revisión de pruebas.

→ 02

Preparar y ejecutar

Datos identificables y verificaciones claras. Guardar logs, trazas y resultados.

→ 03

Conservar evidencia

Artefactos asociados a corrida, caso y prueba. Permiten investigar y reproducir.

> La evidencia respalda qué pasó. La preparación de datos y las condiciones hacen posible volver a probar.

Un rojo necesita diagnóstico

El reporte distingue la causa antes de registrar un defecto de producto.

P

Producto

El sistema contradice un resultado esperado con respaldo validado.

A

Ambiente

Servicio caído, acceso, datos o configuración impiden evaluar el caso.

T

Prueba

El script, selector o preparación falla. Hay que corregir la automatización.

> QA revisa la evidencia. Cada rojo no se convierte automáticamente en bug, y la expectativa no cambia solo para que la prueba pase.

Un bug que Desarrollo puede reproducir

El Markdown del workbook se mantiene como una salida útil. Boards registra el defecto confirmado.

BUG

Cancelación conserva una reserva

Contexto

Versión, ambiente, datos y precondiciones.

Reproducción

Pasos mínimos, esperado respaldado y resultado observado.

Evidencia

Capturas, request/response, logs y consulta autorizada.

Relaciones

Feature, regla, caso, prueba y corrida. Responsable y seguimiento.

Copilot ayuda a redactar. QA confirma el defecto y su clasificación antes del registro.

El cambio empieza en Boards

Una feature registra el cambio esperado. Desarrollo aporta qué cambió y qué puede impactar.

1 **Feature con contexto**
Regla nueva o modificada, dependencias y versión objetivo del sistema.

2 **Actualización en una rama**
Copilot prepara notas técnicas y borrador funcional. Desarrollo aporta la información.

3 **Coordinación y revisión**
El responsable del conocimiento convoca a quienes validan el cambio.

> Una rama permite trabajar sin alterar la publicación vigente. El responsable coordina, pero necesita aportes del equipo.

La revisión aprueba el cambio de conocimiento

Una pull request reúne los cambios y se vincula a la feature.

01

Pull request

Propuesta de cambio en Azure Repos. Revisión técnica y validación funcional.

→ 02

Integración a main

Solo contenido aprobado. Políticas de ramas, revisores y vínculos según el proceso del equipo.

→ 03

Sincronización

El buscador recibe la misma revisión aprobada, con versión de sistema y metadatos.

> PR significa propuesta de cambio para revisión. Main es la rama principal, no la versión que necesariamente está desplegada en QA.

La corrida usa la versión del sistema bajo prueba

La última revisión de main puede describir una funcionalidad que todavía no llegó a QA.

Ejemplo ilustrativo

Desarrollo / main

2.5

Nuevo límite de descuento: 15%.
Documentación aprobada para 2.5.

Ambiente bajo prueba

QA desplegado

2.4

Límite vigente: 10%.
Usar documentación de 2.4.

> Cada corrida fija sistema + ambiente + revisión de conocimiento/docs + revisión de pruebas. Los analistas traen esa revisión, no “latest” sin verificar.

Una regla modificada tiene relaciones

Los vínculos permiten actualizar solo los documentos y las pruebas alcanzados por el cambio.

Feature o bug

Cambio confirmado

→

- Notas técnicas y documentación funcional
- Casos y resultados esperados
- Automatizaciones, datos y verificaciones
- Publicación e índice del buscador

Conservamos evidencia e historial. Un fallo de prueba no autoriza cambiar una regla por cuenta propia.

Un piloto con pocos flujos críticos

Un MVP es una primera versión acotada. Elegimos una aplicación para recorrer el ciclo completo.

1 flujo

Idealmente, una UI o API con validación SQL del resultado.

Preparación concreta

Asignar responsables reales por rol. Reusar Copilot, workbook y Azure existentes.

Comprobar entorno/cliente, permisos, ambiente y tecnología. Habilitar solo los dominios MCP necesarios.

El equipo de automatizaciones participa en la aceptación y ejecución repetible.

> Es el primer MVP propuesto, todavía no una implementación terminada.

Qué tiene que quedar al cerrar el piloto

El producto del MVP es un flujo que el equipo pueda revisar, ejecutar y mantener.

01

Conocimiento y documento

Versionados, con fuentes, validación y pendientes.

02

Casos trazables

Reglas y trabajo vinculados en Test Plans y Boards.

03

Prueba reutilizable

Datos, verificaciones y revisión del equipo.

04

Ejecución y evidencia

Runner en Pipelines y resultados investigables.

05

Defectos confirmados

Reproducción y seguimiento en Boards.

06

Ciclo de mantenimiento

Un cambio actualiza el conjunto relacionado.

La capacidad se mide con trabajo aceptado

El piloto crea una línea de base. Los números se completan con evidencia real.

Tiempo total por automatización aceptada	Preparación, generación, revisión, retrabajo y mantenimiento.
Estabilidad	Fallos intermitentes y causas. Un fallo intermitente pasa o falla sin cambio relevante.
Cobertura crítica	Flujos y reglas críticas con casos y automatizaciones aceptados.
Defectos detectados	Defectos confirmados, evidencia y aporte al proceso de QA.
Recuperación de conocimiento	Fragmentos correctos, completos y de la versión elegida.

> Todavía no hay una línea de base ni retorno de inversión (ROI) demostrado. Escalar depende de los resultados del piloto.

Dos circuitos conectados por la versión

El conocimiento aprobado guía las pruebas. Los cambios y resultados vuelven al conocimiento.

CIRCUITO DE CONOCIMIENTO

Fuentes + Copilot + personas

Notas y documento funcional revisados

Azure Repos · revisión aprobada
Publicación e índice del buscador

CIRCUITO DE PRUEBAS

Copilot + herramientas permitidas

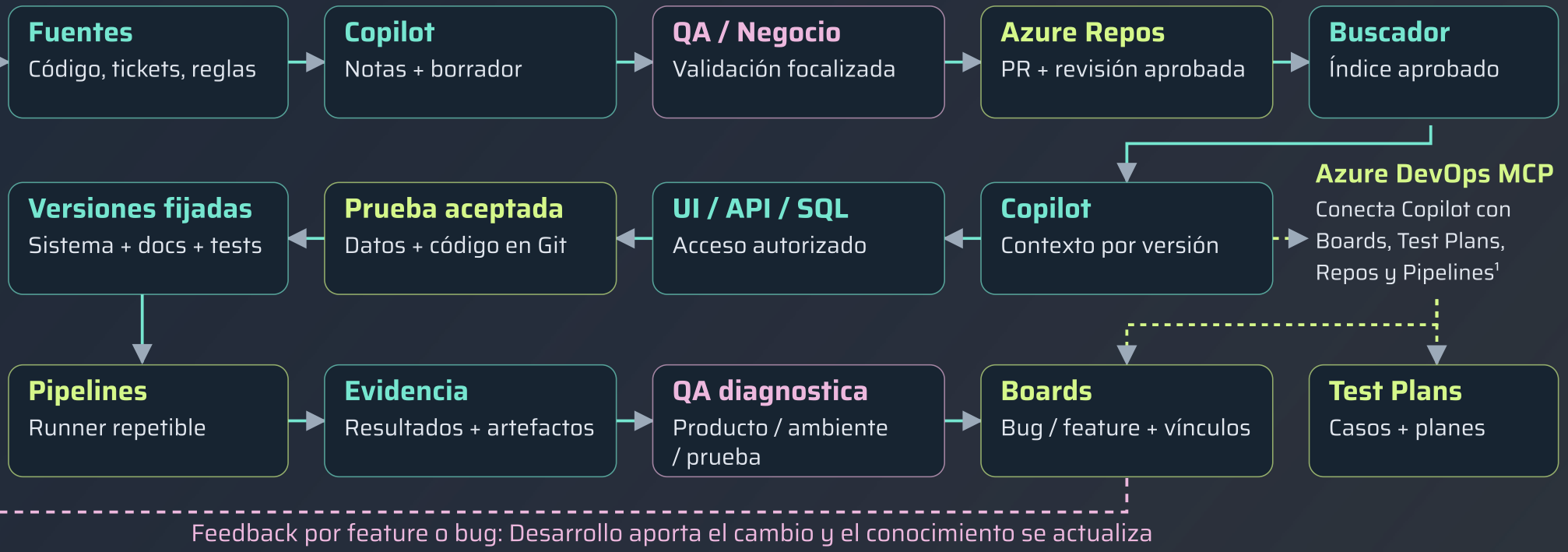
Casos y automatizaciones aceptadas

Test Plans · Pipelines · evidencia
Boards · seguimiento del cambio

> En ambos circuitos: misma versión del sistema y revisiones identificadas. Arquitectura propuesta sobre herramientas existentes.

El ciclo integral de QA agentizada

Una versión aprobada conecta conocimiento, casos, herramientas y resultados.



[1] Integración propuesta. UI web mediante Playwright MCP. API según runner. SQL mediante ejecutor protegido. El buscador recupera la versión elegida.

Base técnica y decisión del piloto

Elegir aplicación, flujos críticos y responsables. Validar las conexiones en el entorno real.

01	**Microsoft Learn · Azure DevOps MCP**	Compatibilidad y conexión oficial
02	**Microsoft · Catálogo de herramientas**	Boards, repos, pipelines y Test Plans
03	**Microsoft · Playwright MCP**	Herramientas para la web
04	**RAGFlow · Configuración**	Metadatos y filtros, verificar versión instalada
05	**Microsoft Learn · Políticas de ramas**	Revisores y vínculos de PR

Herramientas disponibles ≠ integraciones ya instaladas.
Buscador del MVP: confirmar implementación y versión. Notas Markdown conectadas.

Saira · licencia OFL 1.1